EE655 Project: Fast Neural Style Tranfer

Ashish Ahuja
220235
Dept. of EE, IITK
aahuja22@iitk.ac.in

Ayan Gupta
220258
Dept. of EE, IITK
ayangupta22@iitk.ac.in

Devank Saran Prajapati
220339
Dept. of EE, IITK
devanks22@iitk.ac.in

Rahul Ahirwar
220856
Dept. of EE, IITK
rahula22@iitk.ac.in

Abstract

This project addresses the problem of transferring artistic style from one image to another while preserving the semantic content of the original image. We implement a neural style transfer (NST) approach based on convolutional neural networks using a pre-trained VGG19 model. In addition to the standard method, we propose a simplified variant that reduces computational complexity by limiting network depth, reducing input resolution, and decreasing optimization iterations.

Experimental results show that the simplified model achieves a 12x speedup while maintaining visually acceptable outputs. Although the simplified method exhibits reduced stylistic richness, it preserves content effectively and significantly lowers computational cost, making it suitable for time-constrained applications.

1. Introduction

Image style transfer is a fundamental problem in computer vision that lies at the intersection of deep learning and digital art. The goal is to synthesize an image that combines:

- the content of one image
- the style (textures, colors, patterns) of another

Previous work has demonstrated that deep convolutional networks encode both content and style representations, enabling style transfer through optimization.

However, a key limitation of classical NST methods is their high computational cost, as they rely on iterative optimization over deep feature spaces.

1.1. Motivation

1. Standard NST is slow (tens of seconds per image).
2. Not suitable for real-time or large scale applications.

1.2. Contribution

In this project, we:

1. Implement a baseline NST model
2. Propose a simplified variant that:
   - reduceds depth of feature extraction
   - lowers resolution
   - reduces optimization steps
3. Analyze the trade-off b/w speed and quality

2. Methodology

2.1. Baseline Neutral Style Transfer

We use a pre-trained VGG19 convolutional neural network to extract features.

- Content representation: feature maps from deeper layers
- Style representation: Gram matrices of feature maps

The optimization objective is:

$$L_{total} = \alpha L_{content} + \beta L_{style}$$

where content loss is MSE between feature maps and style loss is MSE between gram matrices.

Configuration:

- Image size: 512x512
- Style layers: conv_1 to conv_5
- Content layer: conv_4
- Iterations: 300

2.2. Proposed Simplified Method

To reduce computation we introduce the following modifications:

1. Reduced resolution from 512x512 to 256x256.

2. Fewer layers. conv_1 and conv_2 for style, conv_2 also for content.
3. Fewer itertations. From 300 to 100.
4. Reduced feature complexity.

2.3. Detailed explanation and Tunable parameter

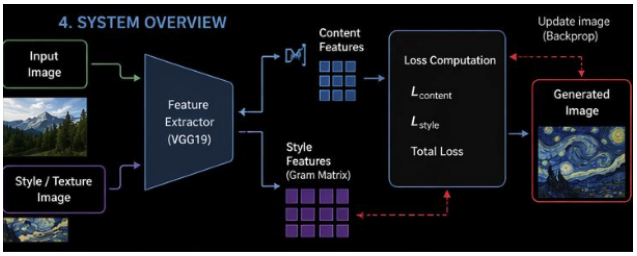

Figure 1. Neural Style Transfer

The implemented program performs Neural Style Transfer (NST) using a pretrained VGG19 Convolutional Neural Network (CNN) as the feature extractor. Since VGG19 has already been trained on the ImageNet dataset, it contains rich hierarchical feature representations that are effective for capturing both image content and artistic style. In our implementation, the CNN weights are frozen, and only the generated image is iteratively optimized.

The content image is passed through deeper layers of the network to preserve its semantic structure, while the style image is processed through multiple convolutional layers to extract texture, color, and pattern information. Style information is represented using Gram matrices computed from feature maps. The final generated image is obtained by minimizing a weighted combination of content loss and style loss:

$$L_{total} = \alpha L_{content} + \beta L_{style}$$

where $L_{content}$ measures similarity with the input image and L_{style} measures similarity with the chosen artistic style.

A key feature of our implementation is that the parameter α is user-tunable. By adjusting α, the user can control how strongly the generated image preserves the original content. A higher value of α results in better structural preservation and weaker stylization, whereas a lower value allows stronger artistic transformation. This makes the model flexible and customizable according to user preference.

Thus, the code provides an efficient NST framework in which users can choose different style images and tune the stylization strength dynamically.

2.4. Expected Impact

- Lower resolution: Faster computation
- Fewer layers: Less expressive style
- Fewer iterations: Faster convergence, lower quality

Figure 2. Style image

Figure 3. Content image for first test

3. Results

We run the models on two test cases for an example comparison. See figures for inputs and outputs.

Figure 6. Content image for second test

Figure 4. Result from the full model

Figure 5. Result from our fast model

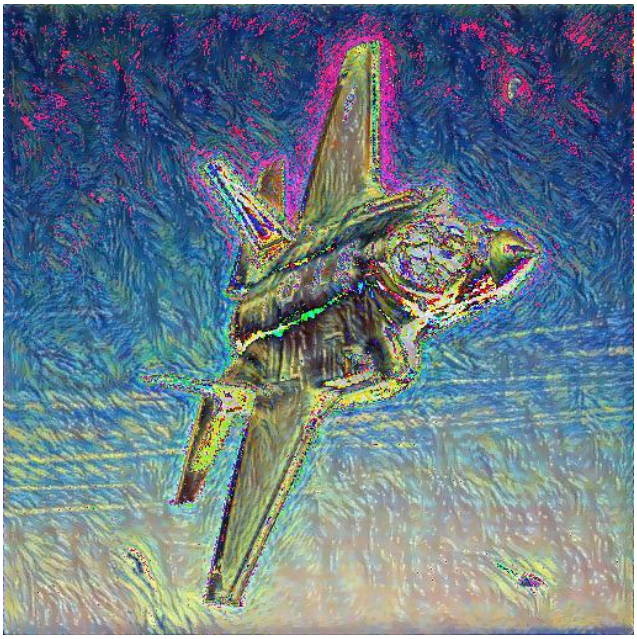

Figure 7. Result from the full model

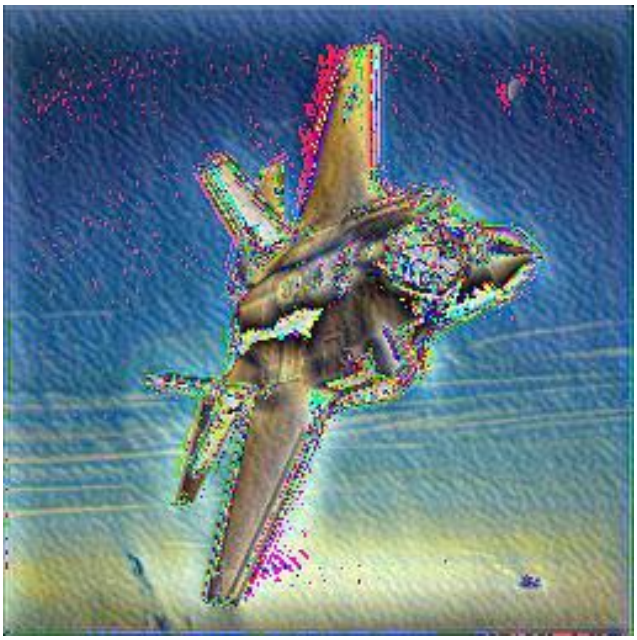

Figure 8. Result from our fast model

3.1. Performance comparison

Table 1. Runtime Comparison Between Full NST and Fast NST

Experiment	Full Model Time (s)	Fast Model Time (s)
Case 1	36.76	3.42
Case 2	48.67	4.06

3.2. Loss comparison

Table 2. Loss Comparison Between Full NST and Fast NST

Experiment	Full Content Loss	Fast Content Loss
Case 1	5.3087	0.3687
Case 2	3.3350	0.1713

3.3. Net speedup

Table 3. Speedup Achieved by Fast NST

Experiment	Speedup (Full / Fast)
Case 1	10.75×
Case 2	11.99×

3.4. Observations

The fast model is significantly quicker due to reduced computation. The fast model also achieves lower content loss, indicating better structural preservtion.

We observed near zero style loss for both models, but this is misleading. From the output images it can clearly be seen that the full model produces richer textures, whereas the fast model produces simpler, less detailed stylization as expected. Stronger artistic patterns and better overall coherence is also observed in the outputs of the full model.

There is slight loss of detail in the fast model, but the results are mostly acceptable.

4. Appendix

Link to github repository.